Ex.No :6 Memory Access Latency with and without Cache

AIM

To write a C program that demonstrates the difference in memory access latency when data i accessed with cache (repeated access) and without cache (random access).

ALGORITHM

1. Declare a large array and initialize it with values.
2. Record the start time.
3. Access the same memory locations repeatedly (cache-friendly access).
4. Record the end time and calculate the elapsed time.
5. Record the start time again.
6. Access memory locations randomly throughout the array (cache-unfriendly access).
7. Record the end time and calculate the elapsed time.
8. Compare the execution times of both cases.
9. Display the latency results.

PROGRAM (C)

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 1000000

int main()
{
   int arr[SIZE];
   long long sum = 0;
   clock_t start, end;
   double cache_time, nocache_time;

   // Initialize array
   for(int i = 0; i < SIZE; i++)
      arr[i] = i;

   // Cache-friendly access
   start = clock();

   for(int j = 0; j < 100; j++)
   {
      for(int i = 0; i < SIZE; i++)
         sum += arr[i];
   }
```

```
    end = clock();
    cache_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    // Cache-unfriendly access
    start = clock();

    for(int j = 0; j < 100; j++)
    {
        for(int i = 0; i < SIZE; i++)
        {
            int index = rand() % SIZE;
            sum += arr[index];
        }
    }

    end = clock();
    nocache_time = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Memory Access Latency Demonstration\n");
    printf("---------------------------------\n");
    printf("Cache-Friendly Access Time   : %f seconds\n", cache_time);
    printf("Random Access Time           : %f seconds\n", nocache_time);

    return 0;
}
```

OUTPUT

```
Memory Access Latency Demonstration
---------------------------------
Cache-Friendly Access Time   : 0.184000 seconds
Random Access Time           : 1.026000 seconds
```

Note: The execution times vary depending on the processor, cache size, compiler, and system configuration.

RESULT

Thus, the C program to demonstrate memory access latency with and without cache was executed successfully.

Ex.No :7 Simulation of a 5-Stage Pipeline

AIM

To simulate a 5-stage instruction pipeline.

ALGORITHM

1. Start the program.
2. Store the instructions in an array.
3. Define the five pipeline stages: IF, ID, EX, MEM, and WB.
4. Display the instructions.
5. For each clock cycle:
    - Determine the stage of each instruction.
    - Display the corresponding pipeline stage.
6. Continue until all instructions complete execution.
7. Stop the program.

PROGRAM

```
#include <stdio.h>
int main()
{
   char *instr[] = {
      "ADD R1,R2,R3",
      "SUB R4,R5,R6",
      "MUL R7,R8,R9"
   };

   char *stage[] = {"IF", "ID", "EX", "MEM", "WB"};

   int cycle, i, s;

   printf("Instructions:\n");
   for(i = 0; i < 3; i++)
      printf("I%d : %s\n", i + 1, instr[i]);

   printf("\nPipeline Execution:\n\n");

   printf("Clock Cycle\tI1\tI2\tI3\n");

   for(cycle = 1; cycle <= 7; cycle++)
   {
      printf("%d\t\t", cycle);

      for(i = 0; i < 3; i++)
```


18

```
        {
            s = cycle - i - 1;

            if(s >= 0 && s < 5)
                printf("%s\t", stage[s]);
            else
                printf("\t");
        }
        printf("\n");
    }

    return 0;
}
```

OUTPUT

Instructions:
I1 : ADD R1,R2,R3
I2 : SUB R4,R5,R6
I3 : MUL R7,R8,R9

Pipeline Execution:

Clock Cycle	I1 (ADD)	I2 (SUB)	I3 (MUL)
1	IF		
2	ID	IF	
3	EX	ID	IF
4	MEM	EX	ID
5	WB	MEM	EX
6		WB	MEM
7			WB

RESULT

Thus, the execution of three instructions (ADD, SUB, and MUL) was successfully simula
using a 5-stage pipeline architecture consisting of IF, ID, EX, MEM, and WB stages, and
corresponding pipeline timing diagram was obtained.

Ex.No :8 Pipeline Hazard Detection and Resolution

AIM

To develop a C program that identifies data hazards in a pipelined processor and resol
them using forwarding and stalling mechanisms.

ALGORITHM

1. Store a sequence of instructions.
2. Compare consecutive instructions.
3. Detect register dependencies.
4. If dependency occurs after a Load instruction, insert a stall.
5. Otherwise, apply forwarding.
6. Display detected hazards and their resolutions.
7. Count the number of forwarding operations and stalls.

PROGRAM

```
#include <stdio.h>
#include <string.h>

typedef struct
{
   char opcode[10];
   char dest[5];
   char src1[5];
   char src2[5];
} Instruction;

int main()
{
   Instruction inst[] =
   {
      {"LW",  "R1", "R2", ""},
      {"ADD", "R3", "R1", "R4"},
      {"SUB", "R5", "R3", "R6"},
      {"MUL", "R7", "R5", "R8"}
   };

   int n = 4;
   int stalls = 0, forwards = 0;

   printf("Instruction Sequence:\n\n");

   for(int i = 0; i < n; i++)
   {
      printf("I%d : %s %s %s %s\n",
```


20

```
            i+1,
            inst[i].opcode,
            inst[i].dest,
            inst[i].src1,
            inst[i].src2);
    }

    printf("\nHazard Detection and Resolution\n");
    printf("--------------------------------\n");

    for(int i = 1; i < n; i++)
    {
       if(strcmp(inst[i-1].dest, inst[i].src1) == 0 ||
         strcmp(inst[i-1].dest, inst[i].src2) == 0)
       {
          printf("\nHazard between I%d and I%d on register %s\n",
               i, i+1, inst[i-1].dest);

          if(strcmp(inst[i-1].opcode, "LW") == 0)
          {
             printf("Resolution : STALL inserted (Load-Use Hazard)\n");
             stalls++;
          }
          else
          {
             printf("Resolution : FORWARDING applied\n");
             forwards++;
          }
       }
    }

    printf("\n--------------------------------\n");
    printf("Total Forwardings : %d\n", forwards);
    printf("Total Stalls      : %d\n", stalls);

    return 0;
}
```

21

OUTPUT

Instruction Sequence:

I1 : LW R1 R2
I2 : ADD R3 R1 R4
I3 : SUB R5 R3 R6
I4 : MUL R7 R5 R8

Hazard Detection and Resolution

Ex.No :9 Interrupt-Driven I/O Simulation

AIM

To simulate Interrupt-Driven Input/Output (I/O) using C programming.

ALGORITHM

1. Start program execution.
2. Simulate CPU performing normal tasks.
3. Generate an interrupt request from an I/O device.
4. CPU stops the current task and invokes the Interrupt Service Routine (ISR).
5. ISR handles the I/O operation.
6. Control returns to the main program.
7. Continue execution of remaining tasks.
8. End the program.

PROGRAM

```
#include <stdio.h>
#include <unistd.h>   // For sleep()

// Interrupt Service Routine
void ISR()
{
   printf("\n*** Interrupt Received ***\n");
   printf("Executing Interrupt Service Routine...\n");
   printf("I/O Operation Completed.\n");
   printf("Returning to Main Program...\n\n");
}

int main()
{
   int i;

   printf("Interrupt-Driven I/O Simulation\n\n");

   for(i = 1; i <= 10; i++)
   {
      printf("CPU executing task %d\n", i);

      sleep(1);

      // Simulate interrupt occurrence
      if(i == 5)
      {
```


23

```
            ISR();
        }
    }

    printf("All CPU tasks completed.\n");

    return 0;
}
```

OUTPUT

```
Interrupt-Driven I/O Simulation

CPU executing task 1
CPU executing task 2
CPU executing task 3
CPU executing task 4
CPU executing task 5

*** Interrupt Received ***
Executing Interrupt Service Routine...
I/O Operation Completed.
Returning to Main Program...

CPU executing task 6
CPU executing task 7
CPU executing task 8
CPU executing task 9
CPU executing task 10

All CPU tasks completed.
```

RESULT

Thus, the simulation of Interrupt-Driven I/O was successfully implemented using C programming.

Ex.No:10 Memory-Mapped I/O Interface

AIM

To simulate a memory-mapped I/O interface for an LED peripheral using C.

PROGRAM

```
#include <stdio.h>

#define LED_ADDRESS 0xFF00

unsigned char memory[65536];   // Simulated memory

void writeMemory(unsigned short address, unsigned char data)
{
   memory[address] = data;

   if(address == LED_ADDRESS)
   {
      if(data == 1)
         printf("LED Status : ON\n");
      else
         printf("LED Status : OFF\n");
   }
}

unsigned char readMemory(unsigned short address)
{
   return memory[address];
}

int main()
{
   printf("Memory-Mapped I/O Simulation\n\n");

   printf("Writing 1 to address 0x%X\n", LED_ADDRESS);
   writeMemory(LED_ADDRESS, 1);

   printf("Writing 0 to address 0x%X\n", LED_ADDRESS);
   writeMemory(LED_ADDRESS, 0);

   printf("Writing 1 to address 0x%X\n", LED_ADDRESS);
   writeMemory(LED_ADDRESS, 1);

   return 0;
}
```

OUTPUT

Memory-Mapped I/O Simulation

Writing 1 to address 0xFF00
LED Status : ON

Writing 0 to address 0xFF00
LED Status : OFF

Writing 1 to address 0xFF00
LED Status : ON

RESULT

Thus, the memory-mapped I/O interface for an LED peripheral was successfully simulated using C. Writing data to the assigned memory address (0xFF00) controlled the LED status (ON/OFF), demonstrating the concept of memory-mapped I/O.